Guía de Actividades Práctico-Experimentales Nro. 005

1. Datos Generales

Asignatura	Estructura de Datos
Ciclo	3ro A / B
Unidad	1: Estructuras de datos estáticas y dinámicas
Integrantes	Matias Encarnación Matias Lavanda Jefferson Sarango Sherman Abarca Julián Merino
Resultado de aprendizaje de la unidad	Listas Doblemente Enlazadas y Pilas: Navegación Bidireccional y Gestión LIFO de Mínimo Costo Energético
Práctica Nro.	005
Título de la Práctica	Listas Doblemente Enlazadas y Pilas: Navegación Bidireccional y Gestión LIFO de Mínimo Costo Energético
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 22 y viernes 23 de octubre 2026
Horario	Jueves 29 de octubre 10:30 – 11:30 Viernes 30 de octubre 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

- Modear e implementar en JavaScript (Node.js) una estructura de Lista Doblemente Enlazada y una Pila (Stack LIFO) para la simulación de un módulo de navegación bidireccional y un sistema transaccional de deshacer/rehacer (*Undo/Redo*) para el proyecto de gestión satelital sustentable.
- Evaluar experimentalmente la eficiencia temporal de inserción/eliminación en extremos frente a la reestructuración por desplazamiento en arreglos nativos.

- Cuantificar el consumo de memoria Heap generado por el almacenamiento de punteros dobles y mediante métricas del motor V8, garantizando el cumplimiento de criterios de Sostenibilidad Computacional (*Green Computing*).

3. Materiales y reactivos:

- Entorno de ejecución Node.js (versión LTS 20.x o superior) instalado.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para consultas a la documentación de estructuras dinámicas.
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.

Computadora personal o de laboratorio:

- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos.
- Memoria RAM: Mínimo 4 GB.

5. Procedimiento / Metodología

Enfoque PID-2025:

- ABP (Aprendizaje Basado en Problemas): Los estudiantes resolverán el reto de diseñar un subsistema de telemetría y auditoría de un satélite sustentable, el cual requiere almacenar eventos en un historial de navegación bidireccional (adelante/atrás) y ejecutar comandos de reversión inmediata de fallos en modo LIFO (*Last In, First Out*).
- ABI (Aprendizaje Basado en Investigación): Investigación empírica sobre la huella espacial de los punteros de 64 bits en la V8 Engine y la justificación matemática de la compensación entre espacio y tiempo (*Space-Time Trade-off*).

Tarea 1: Construcción de la Lista Doblemente Enlazada (Navegación Bidireccional)

1. En la carpeta del proyecto /APE_005_Listas_Pilas, cree el archivo ListaDoble.js.
2. Implemente la clase NodoDoble y la clase ListaDoblementeEnlazada garantizando que las referencias cruzadas se actualicen en tiempo constante O1.

JavaScript

```
/**
 * Representa un Nodo en una Lista Doblemente Enlazada.
 */
class NodoDoble {
  constructor(datos) {
    this.datos = datos; // Carga útil de telemetría satelital
    this.next = null;   // Referencia al siguiente nodo
    this.prev = null;   // Referencia al nodo anterior
  }
}

/**
 * Estructura de Datos Dinámica Doblemente Enlazada.
 */
class ListaDoblementeEnlazada {
  constructor() {
    this.head = null;
    this.tail = null;
    this.longitud = 0;
  }

  /**
   * Inserta un nuevo elemento al final de la lista.
   * Complejidad Temporal: O(1)
   */
  insertarFinal(datos) {
    const nuevoNodo = new NodoDoble(datos);
    if (!this.head) {
      this.head = nuevoNodo;
      this.tail = nuevoNodo;
    } else {
      this.tail.next = nuevoNodo;
      nuevoNodo.prev = this.tail;
      this.tail = nuevoNodo;
    }
    this.longitud++;
    return nuevoNodo;
  }
```

```
 /**
  * Elimina el último elemento de la lista.
  * Complejidad Temporal: O(1)
  */
 eliminarFinal() {
  if (!this.tail) return null;
  const nodoEliminado = this.tail;
  if (this.head === this.tail) {
   this.head = null;
   this.tail = null;
  } else {
   this.tail = this.tail.prev;
   this.tail.next = null;
  }
  nodoEliminado.prev = null;
  this.longitud--;
  return nodoEliminado.datos;
 }
}

module.exports = { NodoDoble, ListaDoblementeEnlazada };
```

Tarea 2: Implementación de la Pila Transaccional (Stack LIFO) sobre Punteros

1. Cree el archivo PilaTransaccional.js.
2. Desarrolle la Pila de comandos satelitales apoyada en nodos enlazados para evitar la relocalización de memoria de los arreglos dinámicos al hacer push o pop.

JavaScript

```
const { ListaDoblementeEnlazada } = require('./ListaDoble');

/**
 * Pila (Stack) orientada a Sostenibilidad Computacional.
 * Operaciones fundamentales Push y Pop en O(1) estricto.
 */
class PilaTransaccional {
 constructor() {
  this.lista = new ListaDoblementeEnlazada();
 }

 /**
  * Apila un comando de telemetría en el tope.
  * Complejidad Temporal: O(1)
  */
 push(comando) {
  this.lista.insertarFinal(comando);
 }
```

```
 /**
  * Desapila y retorna el comando en el tope (Undo).
  * Complejidad Temporal: O(1)
  */
 pop() {
   return this.lista.eliminarFinal();
 }

 /**
  * Retorna la cantidad de elementos almacenados.
  */
 obtenerTamano() {
   return this.lista.longitud;
 }
}

module.exports = PilaTransaccional;
```

Tarea 3: Experimento Comparativo (Benchmark CPU & Memory Heap)

1. Cree el archivo BenchmarkNavegacion.js.
2. Implemente un script que ejecute $N = 500.000$ inserciones y extracciones al inicio/final comparando un Array nativo de JavaScript frente a la PilaTransaccional.

JavaScript

```
const PilaTransaccional = require('./PilaTransaccional');

function obtenerMemoriaMB() {
  return (process.memoryUsage().heapUsed / 1024 / 1024).toFixed(2);
}

const N = 500000;
console.log(`=== BENCHMARK DE RENDIMIENTO (N = ${N}) ===\n`);

// 1. EVALUACIÓN CON ARREGLO TRADICIONAL (unshift / shift)
if (global.gc) global.gc();
const memInicialArray = process.memoryUsage().heapUsed;
console.time('Tiempo Array Nativo (unshift/shift)');

const arregloNativo = [];
for (let i = 0; i < N; i++) {
  arregloNativo.unshift({ id: i, log: 'Evento Satelital' }); // O(n)
}
for (let i = 0; i < N; i++) {
  arregloNativo.shift(); // O(n)
}
```

```
console.timeEnd('Tiempo Array Nativo (unshift/shift)');
const memFinalArray = process.memoryUsage().heapUsed;
const deltaMemArray = ((memFinalArray - memInicialArray) / 1024 / 1024).toFixed(2);
console.log(`Impacto de Memoria Heap Array: ${deltaMemArray} MB\n`);

// 2. EVALUACIÓN CON PILA DE PUNTEROS DINÁMICOS
if (global.gc) global.gc();
const memInicialPila = process.memoryUsage().heapUsed;
console.time('Tiempo Pila Transaccional O(1)');

const pilaSatelital = new PilaTransaccional();
for (let i = 0; i < N; i++) {
  pilaSatelital.push({ id: i, log: 'Evento Satelital' }); // O(1)
}
for (let i = 0; i < N; i++) {
  pilaSatelital.pop(); // O(1)
}

console.timeEnd('Tiempo Pila Transaccional O(1)');
const memFinalPila = process.memoryUsage().heapUsed;
const deltaMemPila = ((memFinalPila - memInicialPila) / 1024 / 1024).toFixed(2);
console.log(`Impacto de Memoria Heap Pila: ${deltaMemPila} MB\n`);
```

3. Ejecute el script con la bandera de recolección de basura:

   Bash

   ```
   node --expose-gc BenchmarkNavegacion.js
   ```

6. Resultados Esperados

El estudiante evidenciará a través Registro de ejecución en la consola donde se evidencia que las operaciones de inserción y extracción masiva mediante la PilaTransaccional reducen drásticamente el tiempo de ejecución en milisegundos ms en comparación con unshift() y shift() del arreglo nativo, demostrando la diferencia matemática.

Resultados de node --expose-gc BenchmarkNavegacion.js:

```
PS C:\Users\nicof\OneDrive\Desktop\Archivos\Estructura de Datos (Grupal)\APE_005_Listas_Pilas> node
-expose-gc BenchmarkNavegacion.js
=== BENCHMARK DE RENDIMIENTO (N = 500000) ===

Tiempo Array Nativo (unshift/shift): 4:03.654 (m:ss.mmm)
Impacto de Memoria Heap Array: 26.50 MB

Tiempo Pila Transaccional O(1): 66.61ms
Impacto de Memoria Heap Pila: 42.03 MB

PS C:\Users\nicof\OneDrive\Desktop\Archivos\Estructura de Datos (Grupal)\APE_005_Listas_Pilas>
```

7. Preguntas de Control:

Responda fundamentando sus argumentos bajo una perspectiva lógica, algorítmica y filosófica:

- En el contexto de las estructuras de datos dinámicas, ¿cuál es la "sustancia" de un nodo en una Lista Doblemente Enlazada en comparación con la de una celda de un arreglo contiguo, y cómo modifica el puntero anterior (prev) su naturaleza constitutiva?

  En un arreglo contiguo, los elementos ocupan bloques de memoria que estan estrictamente ordenados y que se encuentran empaquetados uno tras otro. En contraste un nodo en una lista que se encuentra doblemente enlazada es una unidad dinamica que se encuentra dispersa en el heap, cuya sustancia integra tanto la carga util de los datos como las referencias direccionales next y prev. Especificamente el puntero prev modifica su naturaleza constitutiva al otorgarle la capacidad de retroceder, transformando una cadena unidireccional en una red bidirecional que permite navegar en ambos sentidos sin depender de indices numericos fijos.

- ¿Por qué la operación de inserción y extracción LIFO en una Pila implementada mediante punteros dobles alcanza la causa eficiente de la inmediatez O1, superando el trabajo mecánico de reubicación física de celdas que realiza el motor V8 en un arreglo nativo?

  Esto impacta directamente en la eficiencia, ya que mientras que los arreglos nativos fuerzan al motor V8 a mover fisicamente todos los elementos con un costo lineal O(n) mediante operaciones como shift o unshift, una pila con punteros dobles ejecuta sus inserciones y extracciones actualizando solo los enlaces de los extremos, logrando una inmediatez estricta de O(1) sin relocalizacion mecanica de celdas.

- ¿Cuál es el fin último (*telos*) de la navegación bidireccional en un historial de operaciones o sistema de transacciones *Undo/Redo*, y en qué medida optimiza la estabilidad energética del sistema de cómputo?

  El fin ultimo (telos) de esta navegacion bidireccional es potenciar historiales transaccionales fluidos, ideales para sistemas de tipo Undo/Redo. Al evitar las reubicaciones de memoria, el sistema requiere muchos menos ciclos de CPU, lo que optimiza la estabilidad energetica y respalda directamente los principios de Green Computing.

- Aristóteles sostenía que nada existe sin una forma y materia dada. Si para ganar flexibilidad bidireccional se requiere duplicar las referencias de memoria prev y next, ¿cuándo deja el gasto de memoria Heap de ser una virtud de diseño para convertirse en un exceso ontológico (*overhead*)?

  Aunque duplicar los punteros prev y next aporta una gran flexibilidad de diseño en la memoria Heap, esto deja de ser una virtud para convertirse en un exceso ontològico u overhead cuando el consumo de espacio de las referencias supera desproporcionadamente al de los datos reales, o cuando la logica de la aplicación no justifica la bidirecionalidad.

- Considerando la conservación de energía como principio técnico y ético, ¿de qué manera la eliminación instantánea de referencias sin desplazamiento de elementos mitiga el trabajo electromagnético de la CPU?

  Mover bloques de memoria obliga a los circuitos y buses a un trasiego masivo de electrones mediante una conmutacion intensiva de transistores. En la lista doble, en cambio la eliminacion de un nodo se resuelve reconectando los enlaces adyacentes para dejar que el Garbage collector limpie la memoria, evitando el desplazamiento fisico de los elementos, mitigando notablemente el trabajo de la CPU.

8. Evaluación

Criterio Evaluado	Evidencia Esperada	Puntaje Máximo
Análisis Green Computing	Justificación técnica del ahorro de CPU vs gasto de RAM.	2.0
Arquitectura de Nodos	Implementación correcta de la referencia bidireccional.	4.0
Navegación Fluida	Métodos next() y prev() funcionales con complejidad .	2.0
Defensa Técnica	Explicación video del impacto de los punteros en la energía.	2.0
TOTAL		**10.0**

9. Declaración de IA

Para la realización del trabajo presente se hizo uso del modelo de IA Gemini 3.1 Pro haciendo uso del siguiente prompt: "Dame un resumen claro de los conceptos que debo saber para responder las preguntas de control de esta guía, necesito la info base para yo mismo armar las respuestas".

10. Bibliografía

Liste las fuentes utilizadas. Use normas **IEEE**. Pueden ser libros, manuales, artículos, tutoriales académicos, documentación oficial de software.

[1] B. Goetz, Java Concurrency in Practice. Addison-Wesley, 2022.

[2] P. W. Bible and L. Moser, An Open Guide to Data Structures and Algorithms. PALNI Open Press, 2023.

[3] J. Bloch, Effective Java, 3rd ed. Boston, MA: Addison-Wesley, 2018.

1. Elaboración y Aprobación

- Elaborado por: nombre del Docente responsable de la práctica.
- Aprobado por: nombre del Director de Carrera.

Elaborado por	Cristian R. Narváez Guillén **Docente**	
Aprobado por	Edison L Coronel Romero **Director de Carrera**	